

IT- 603 Lab 2: C++ Compilation, Debugging, Build Systems, and Version Control

Kunal Adwani | 202612050 | [Kunal-63/202612050-IT603](#)

1 Part A: GCC/G++ Compilation Process

What to do

- Write a simple C++ program (e.g., a program that computes factorial or checks primality).
- Compile it using g++ and run the executable.
- Explore each stage of compilation separately using compiler flags:
 - Preprocessing – generate the preprocessed output (-E)
 - Compilation – generate assembly code (-S)
 - Assembly – generate the object file (-c)
 - Linking – produce the final executable
- Note down the file produced at each stage and briefly describe what changed.
- Try compiling with different optimization levels (-O0, -O2) and compare.

```
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ cat factorial.cpp
#include <iostream>

int factorial(int number) {
    int result = 1;
    for (int value = 2; value <= number; ++value) {
        result *= value;
    }
    return result;
}

int main() {
    int number;
    std::cout << "Enter a non-negative integer: ";
    std::cin >> number;

    if (number < 0) {
        std::cout << "Factorial is not defined for negative numbers.\n";
        return 1;
    }

    std::cout << number << "! = " << factorial(number) << '\n';
    return 0;
}

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ g++ factorial.cpp -o factorial

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ ./factorial
Enter a non-negative integer: 5
5! = 120

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ ls
202612050.docx  factorial.cpp  factorial.exe  '~$2612050.docx'
```

Stage 1 – Preprocessing

- #include <iostream> is expanded.
- Preprocessor directives are handled.
- Macros are expanded.
- The actual C++ source is converted into a much larger preprocessed source file.

```
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ g++ -E factorial.cpp -o factorial.i
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ less factorial.i
```

Stage 2 – Compilation

- C++ source is converted into assembly language.
- The compiler performs syntax and semantic analysis.
- Optimizations may be performed depending on the optimization flag.

```
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ g++ -S factorial.cpp -o factorial.s
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ ls
202612050.docx  factorial.cpp  factorial.exe  factorial.i  factorial.s  '~$2612050.docx'
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$
```

Stage 3 – Assembly

- Assembly code is converted into machine/object code.
- The result is an object file.
- It is not yet a complete executable because library functions such as iostream still need to be linked.

```
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ g++ -c factorial.cpp -o factorial.o
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ ls
202612050.docx  factorial.cpp  factorial.exe  factorial.i  factorial.o  factorial.s  '~$2612050.docx'
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ |
```

Stage 4 - Linking

- The linker combines your object code with required libraries.
- References to functions such as cout and cin are resolved.
- A complete executable program is produced.

```
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ g++ factorial.o -o factorial
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ ./factorial.exe
Enter a non-negative integer: 5
5! = 120
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$
```

```

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ g++ -O0 factorial.cpp -o factorial_00

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ g++ -O2 factorial.cpp -o factorial_02

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ g++ -O0 -S factorial.cpp -o factorial_00.s
g++ -O2 -S factorial.cpp -o factorial_02.s

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ diff factorial_00.s factorial_02.s
-bash: diff: command not found

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ ls -lh
total 1.7M
-rw-r--r-- 1 kunal kunal 23K Aug 19 16:33 202612050.docx
-rw-r--r-- 1 kunal kunal 502 Aug 19 09:27 factorial.cpp
-rwxr-xr-x 1 kunal kunal 112K Aug 19 16:43 factorial.exe
-rw-r--r-- 1 kunal kunal 1.3M Aug 19 16:37 factorial.i
-rw-r--r-- 1 kunal kunal 2.0K Aug 19 16:42 factorial.o
-rw-r--r-- 1 kunal kunal 2.7K Aug 19 16:41 factorial.s
-rwxr-xr-x 1 kunal kunal 112K Aug 19 16:44 factorial_00.exe
-rw-r--r-- 1 kunal kunal 2.7K Aug 19 16:45 factorial_00.s
-rwxr-xr-x 1 kunal kunal 112K Aug 19 16:44 factorial_02.exe
-rw-r--r-- 1 kunal kunal 3.2K Aug 19 16:45 factorial_02.s
-rw-r--r-- 1 kunal kunal 162 Aug 19 16:31 '~$2612050.docx'

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ |

```

With -O0, the compiler performs minimal optimization, resulting in simpler and generally larger assembly code. With -O2, the compiler applies more optimizations such as reducing unnecessary instructions and improving execution efficiency. Therefore, the generated assembly can differ between the two optimization levels.

2 Part B: Debugging Using GDB

What to do

- Take a C++ program with a deliberate bug (e.g., an off-by-one error or a segmentation fault from an out-of-bounds array access).
- Compile it with debugging symbols enabled (-g).
- Load the program in GDB and:
 - Set breakpoints at relevant functions/lines.
 - Step through the program line by line.
 - Inspect variable values at different points.
 - Use a watchpoint to track when a variable changes.
 - Identify and fix the bug.
- Record the sequence of GDB commands used and the final fix.

```

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part b
$ g++ -g debug.cpp -o debug

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part b
$ ./debug
i = 0, sum = 10
i = 1, sum = 30
i = 2, sum = 60
i = 3, sum = 100
i = 4, sum = 150
i = 5, sum = 168
Final Sum = 168

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part b
$ gdb ./debug
GNU gdb (GDB) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-w64-mingw32".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./debug...
(gdb) b main
Breakpoint 1 at 0x1400014ed: file debug.cpp, line 5.
(gdb) r
Starting program: C:\Users\kunal\Desktop\DAU\Sem 1\Introduction to Programming (IT603)\202612050-IT603\Lab 2\part b\debug.exe
[New Thread 30180.0x2780]
[New Thread 30180.0xa24]
[New Thread 30180.0x520c]

Thread 1 hit Breakpoint 1, main () at debug.cpp:5
5         int arr[5] = {10, 20, 30, 40, 50};
(gdb) l
1         #include <iostream>
2         using namespace std;
3
4         int main() {
5             int arr[5] = {10, 20, 30, 40, 50};
6             int sum = 0;
7
8             for (int i = 0; i < 5; i++) {

```

```

2      using namespace std;
3
4      int main() {
5          int arr[5] = {10, 20, 30, 40, 50};
6          int sum = 0;
7
8          for (int i = 0; i <= 5; i++) {
9              sum += arr[i];
10             cout << "i = " << i << ", sum = " << sum << endl;
(gdb) n
6          int sum = 0;
(gdb) n
8          for (int i = 0; i <= 5; i++) {
(gdb) n
9              sum += arr[i];
(gdb) n
10             cout << "i = " << i << ", sum = " << sum << endl;
(gdb) print i
$1 = 0
(gdb) watch sum
Hardware watchpoint 2: sum
(gdb) c
Continuing.
i = 0, sum = 10

Thread 1 hit Hardware watchpoint 2: sum

Old value = 10
New value = 30
main () at debug.cpp:10
10      cout << "i = " << i << ", sum = " << sum << endl;
(gdb) c
Continuing.
i = 1, sum = 30

Thread 1 hit Hardware watchpoint 2: sum

Old value = 30
New value = 60
main () at debug.cpp:10
10      cout << "i = " << i << ", sum = " << sum << endl;
(gdb) p i
$2 = 2
(gdb) c
Continuing.
i = 2, sum = 60

Thread 1 hit Hardware watchpoint 2: sum

Old value = 60
New value = 100
main () at debug.cpp:10
10      cout << "i = " << i << ", sum = " << sum << endl;
(gdb) c
Continuing.
i = 3, sum = 100

Thread 1 hit Hardware watchpoint 2: sum

Old value = 100
New value = 150
main () at debug.cpp:10
10      cout << "i = " << i << ", sum = " << sum << endl;
(gdb) |

```

```

10      cout << "i = " << i << ", sum = " << sum << endl;
(gdb) quit
A debugging session is active.

    Inferior 1 [process 30180] will be killed.

Quit anyway? (y or n) y
kunal@TedMosby UCR764 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part b
$ nano debug.cpp
kunal@TedMosby UCR764 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part b
$ g++ -g debug.cpp -o debug
kunal@TedMosby UCR764 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part b
$ ./debug
i = 0, sum = 10
i = 1, sum = 30
i = 2, sum = 60
i = 3, sum = 100
i = 4, sum = 150
Final Sum = 150
kunal@TedMosby UCR764 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part b
$

```

3 Part C: Multi-file Programs

What to do

- Split a single-file program into multiple files: separate header (.h) and source (.cpp) files for at least one class or module, plus a main.cpp.
- Use header guards (or #pragma once) correctly.
- Compile the multi-file program manually using g++, first generating object files for each source file and then linking them together.
- Explain why splitting code into multiple files is useful for larger projects.

```
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ nano student.h

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ nano student.cpp

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ nano main.cpp

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ ls
main.cpp  student.cpp  student.h

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ g++ -c student.cpp -o student.o

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ g++ -c main.cpp -o main.o

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ ls
main.cpp  main.o  student.cpp  student.h  student.o

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ g++ main.o student.o -o student_program

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ ls
main.cpp  main.o  student.cpp  student.h  student.o  student_program.exe

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ ./student_program
Name: Kunal
Marks: 85
Result: Passed

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$
```

Why use multiple files?

- Better organization
- Easier maintenance
- Code reusability
- Faster compilation
- Team development

4 Part D: Makefiles

What to do

- Write a Makefile for the multi-file program from Part C.
- The Makefile should include:
 - Rules to compile each source file into an object file.
 - A rule to link all object files into the final executable.
 - A clean target to remove generated files.
- Use variables (e.g., CXX, CXXFLAGS, OBJS) instead of hardcoding values.
- Run make and make clean to confirm the build works end-to-end.

```
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ nano Makefile

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ make
make: 'student_program' is up to date.

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ ./student_program.exe
Name: Kunal
Marks: 85
Result: Passed

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ make
g++ -Wall -g -c student.cpp -o student.o
g++ main.o student.o -o student_program

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ ./student_program.exe
Name: Kunal
Student Marks: 85
Result: Passed

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c
$ cd ..

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2
$ cd part\ c\ &\ d/

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d
$ make clean
rm -f main.o student.o student_program

kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d
$
```

```
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d
$ cat Makefile
CXX = g++
CXXFLAGS = -Wall -g

TARGET = student_program
OBJS = main.o student.o

$(TARGET): $(OBJS)
    $(CXX) $(OBJS) -o $(TARGET)

main.o: main.cpp student.h
    $(CXX) $(CXXFLAGS) -c main.cpp -o main.o

student.o: student.cpp student.h
    $(CXX) $(CXXFLAGS) -c student.cpp -o student.o

clean:
    rm -f $(OBJS) $(TARGET)
kunal@TedMosby UCRT64 /c/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d
$
```

Source Code of Makefile - [202612050-IT603/Lab 2/part c d/Makefile at master · Kunal-63/202612050-IT603](#)

5 Part E: Introduction to Git and Version Control

What to do

- Initialize a Git repository for the project from Parts C and D.
- Create a .gitignore file to exclude compiled binaries and object files.
- Make an initial commit, then:
 - Make a small code change and commit it separately.
 - View the commit history using git log.
 - Create a new branch, make a change on it, and merge it back into the main branch.
- Briefly note the purpose of git status, git diff, and git checkout based on your usage.

```
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (master)
$ git init
git branch -M main
Initialized empty Git repository in C:/Users/kunal/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d/.git/
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ nano .gitignore
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ cat .gitignore
*.o
*.exe
student_program
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        Makefile
        main.cpp
        student.cpp
        student.h

nothing added to commit but untracked files present (use "git add" to track)
gg
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git add .
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git commit -m "Initial commit"
[main (root-commit) 7605c2a] Initial commit
5 files changed, 75 insertions(+)
create mode 100644 .gitignore
create mode 100644 Makefile
create mode 100644 main.cpp
create mode 100644 student.cpp
create mode 100644 student.h
```

```
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ nano main.cpp
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git diff
diff --git a/main.cpp b/main.cpp
index 79fd2d7..9a502e6 100644
--- a/main.cpp
+++ b/main.cpp
@@ -4,7 +4,7 @@
 using namespace std;

 int main() {
-    Student s("Kunal", 85);
+    Student s("Kunal Adwani", 85);

    s.display();
}
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git add .
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git commit -m "Updated Student Name"
[main 93cb2b1] Updated Student Name
1 file changed, 1 insertion(+), 1 deletion(-)
kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git log --oneline
93cb2b1 (HEAD -> main) Updated Student Name
7605c2a Initial commit
```



```

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git checkout -b passing_marks
Switched to a new branch 'passing_marks'

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (passing_marks)
$ nano student.cpp

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (passing_marks)
$ git diff
diff --git a/student.cpp b/student.cpp
index 6f4fd4c..f074a60 100644
--- a/student.cpp
+++ b/student.cpp
@@ -14,5 +14,5 @@ void Student::display() {
 }

bool Student::isPassed() {
-   return marks >= 40;
+   return marks >= 35;
}
\ No newline at end of file

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (passing_marks)
$ git add .

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (passing_marks)
$ git commit -m "Updated Passing Marks"
[passing_marks 50d23b6] Updated Passing Marks
1 file changed, 1 insertion(+), 1 deletion(-)

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (passing_marks)
$ git checkout main
Switched to branch 'main'

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git merge passing_marks
Updating 93cb2b1..50d23b6
Fast-forward
 student.cpp | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git log --oneline
50d23b6 (HEAD -> main, passing_marks) Updated Passing Marks
93cb2b1 Updated Student Name
7605c2a Initial commit

kunal@TedMosby MINGW64 ~/Desktop/DAU/Sem 1/Introduction to Programming (IT603)/202612050-IT603/Lab 2/part c & d (main)
$ git status
On branch main
nothing to commit, working tree clean

```

git status shows changed, staged, and untracked files.

git diff shows unstaged code changes.

git checkout switches branches; **git checkout -b** creates and switches to a new branch.

GitHub Repo Link - [Kunal-63/202612050-IT603](#)